Jamaa Next Door

A PLATFORM TO UNIFY MUSLIMS

Designing a safer way for Muslims to find congregational prayer nearby.

PROJECT PORTFOLIO

Program: Augmented and Virtual Reality Design

Elective: Experimental XR Interactions & Interfaces

Summer Semester – 2026

Tutored by David Bachmann

Prepared by

Mohitur Rahman Zidan

HOW THIS PORTFOLIO IS ORGANIZED

Contents

This document presents both the finished prototype and the path that produced it. It is written for a reader who has not previously encountered the project.

01 / PROJECT OVERVIEW AND MOTIVATION

Project Overview

Jamaa Next Door is a location-aware Android prototype that helps Muslims discover or organize a nearby congregational prayer when attending a mosque is not practical.

Find | Create | Join | Pray together

The central idea

A user can see upcoming Jama'ahs in the surrounding area, review the prayer and countdown, open a details sheet, and join. If no suitable gathering exists, the user can create one by selecting the prayer, time, location and a location photo, then reviewing and confirming it.

What makes it different

The product is deliberately short-lived and local. A Jama'ah is sorted by its scheduled time and ends automatically when that time arrives. The interface emphasizes what matters in the moment: which prayer, where approximately, how long remains, and whether the user is participating.

Design intention

The prototype is not intended to replace mosques. It supports the gaps between fixed mosque schedules and everyday life: work, classes, meetings, travel, unfamiliar neighborhoods and temporary accommodation.

PROTOTYPE OUTCOME

A standalone APK now installs directly on Android without Expo Go and connects to Supabase for authentication, profile data and Jama'ah publication.

01 / PROJECT OVERVIEW AND MOTIVATION

Why I Chose This Topic

The project began with a personal experience rather than a technology choice.

A difficult first experience in Germany

When I first came to Germany, I lived for two months in an area where there was no mosque nearby. In my religious practice, praying in congregation five times a day is an important obligation. Being unable to do that bothered me throughout the time I lived there.

People were present, but connection was missing

There were many Muslims in the area, yet there was no simple way for us to find one another at prayer time. The problem was not necessarily the absence of people; it was the absence of awareness, coordination and trust. Seeing that lack of unity affected me and stayed in my mind.

Why Germany is relevant

Many places in Germany do not have easy access to a mosque or religious center. Structured work, classes and meetings can also make people miss the mosque's scheduled Jama'ah. Travelers and newcomers face the same uncertainty. Jamaa Next Door explores whether a small local gathering can be organized safely at a later suitable time.

> I wanted to turn the feeling of being religiously isolated into a practical way for nearby people to find one another.

02 / PROBLEM, USERS, GOALS AND SCOPE

Problem and Users

The visible problem is missing congregational prayer. Underneath it are four connected barriers that the product must address together.

Access

A mosque may be too far away, closed, unknown to a newcomer or unreachable during a short break.

Timing

A person may miss the mosque's Jama'ah because of work, classes, meetings, travel or changing schedules.

Awareness

Other Muslims may be nearby, but nobody knows who is available or planning to pray.

Trust

Location sharing, unfamiliar hosts and identity checks create genuine safety and privacy concerns.

Primary user situations

The prototype is designed around workers, students, travelers, newcomers and local hosts. These are situations rather than rigid personas: the same person may be a host near home, a participant near work and a newcomer while traveling.

Core design question

How might an app make a nearby Jama'ah discoverable quickly without exposing exact private locations or asking users to trust strangers blindly?

02 / PROBLEM, USERS, GOALS AND SCOPE

Goals and Prototype Scope

The scope was kept focused on proving the central coordination loop rather than building a complete community platform.

What the prototype must prove

- A user can authenticate and complete a basic profile.
- Upcoming Jama'ahs can be discovered and ordered by time.
- A host can create, review and publish a Jama'ah.
- A participant can inspect details and join or leave.
- Sensitive location details can be revealed conditionally.
- The gathering disappears when its scheduled time arrives.

Deliberate boundaries

- The app does not replace mosque leadership or religious guidance.
- It does not claim that identity or selfie checks remove all risk.
- The current work is a prototype, not a fully staffed moderation service.
- Advanced community, donation and organization features are outside this document.
- Live operation still depends on correctly configured external services.

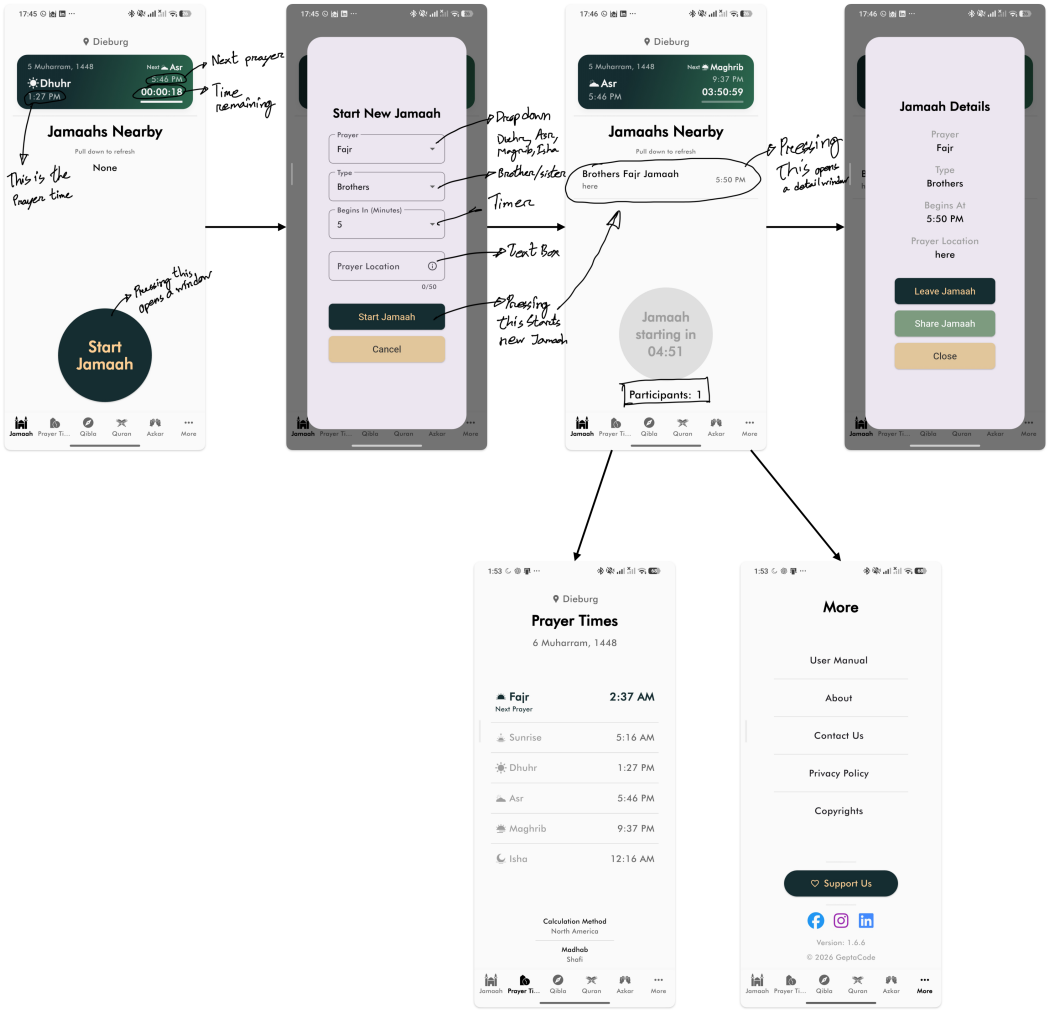
SUCCESS CRITERION

A first-time reader should understand the flow in one sentence: find a nearby prayer, review the details, join safely, or create one when nothing suitable exists.

03 / CONCEPT DEVELOPMENT AND DECISIONS

Interaction Flow Reference

The App Interface and Interaction PDF recorded a comparable app as an annotated flow. I used it to study transitions, action placement and information order before adapting the behavior to Jamaa Next Door.



Annotated reference flow. Source: Research/App Interface and Interaction.pdf.

What the reference made visible

The sequence connects a prominent home action to a structured creation form, returns the new gathering to the nearby list and opens a details view with contextual actions. The handwritten arrows and notes capture the behavior I wanted to understand, not a visual style to copy.

USE OF REFERENCE MATERIAL

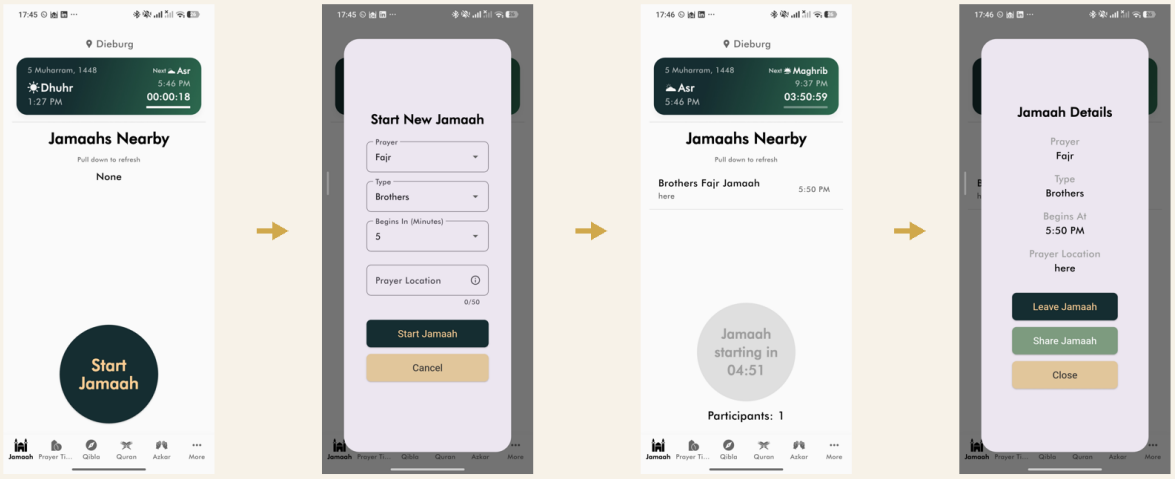
These screenshots belong to the observed reference app and are included as process evidence, not as Jamaa Next Door screens.

03 / CONCEPT DEVELOPMENT AND DECISIONS

From Reference Flow to Product Decisions

I translated the observed interaction pattern into Jamaa Next Door while changing the ownership, privacy and lifecycle rules to fit this project.

REFERENCE INTERACTION SEQUENCE



1 Home and entry action

2 Create Jama'ah form

3 Published nearby card

4 Details and actions

REFERENCE PATTERN	JAMAA NEXT DOOR ADAPTATION
Prominent home action	Keep creation reachable from the Jama'ah home without hiding discovery.
Structured creation form	Collect prayer, time, location and a required photo, then review before publication.
Return to nearby list	Refresh the time-sorted home so the new Jama'ah appears immediately.
Contextual details	Use a bottom sheet with role-based Join, Leave, Cancel, Share and Close actions.
Persistent navigation	Keep supporting tools available without mixing their state into creation.

04 / PRIVACY, SECURITY AND ETHICS

Privacy Risk Landscape

Privacy is not an additional feature in Jamaa Next Door. The central interaction involves strangers, time, identity and physical location, so privacy decisions shape the product architecture.

Risk	Possible harm	Priority
Exact location	A private home or meeting place could be exposed to unrelated users.	High
Location photo	An image may reveal entrances, names, faces or identifying details.	High
Selfie verification	Biometric-like identity material creates storage, access and liability risks.	High
Messages	Moderation can feel like surveillance if readers and conditions are unclear.	Medium
Push notifications	Tokens and notification text can disclose participation or religious activity.	Medium
Abuse and reports	Bad actors may create misleading gatherings or target participants.	High

Privacy principles

Collect only what the core interaction needs; keep sensitive media private; enforce rules on the server rather than only in the interface; explain who can access information; and record privileged actions where accountability is required.

04 / PRIVACY, SECURITY AND ETHICS

Safeguards in the Prototype

The prototype applies layered safeguards. No single measure is treated as sufficient on its own.

Area	Current safeguard
Location	Public discovery returns approximate information. Exact address and location media are restricted to the host or permitted active participants.
Storage	Selfie verification and Jama'ah location images use private Supabase Storage buckets with authenticated, ownership-based policies.
Database	Row Level Security is enabled on application tables. Application functions require authenticated users, while privileged jobs are restricted to trusted server roles.
Participation	Joining is separated from browsing. The server, not only the screen, decides whether a user may join and see protected details.
Accountability	Reports, blocks, notifications, attendance and deletion requests have dedicated data structures rather than being mixed into screen state.
Lifecycle	Cancelled and concluded Jama'ahs are excluded from discovery, and due gatherings are ended by a restricted scheduled operation.

RESIDUAL RISK

Technical controls reduce exposure, but they do not create complete trust. Clear consent, visible policy, moderation responsibility and real user testing are still necessary before public use.

04 / PRIVACY, SECURITY AND ETHICS

Ethics and Transparency

Teacher feedback highlighted areas where transparency is as important as implementation. These questions are documented rather than hidden behind a polished interface.

Chat moderation

The product must state whether messages can be reviewed, who can review them, whose messages are accessible and under which reported circumstances. Moderation should create informed trust, not silent inspection.

Selfie verification

A professional third-party provider may be more trustworthy than building a verification operation internally. This would reduce direct handling of sensitive images, but provider terms, cost and data location still require scrutiny.

Age assurance

A checkbox confirming the user is 18 is only a declaration. It should not be presented as strong identity or age verification.

Language access

Arabic is an obvious language candidate. Right-to-left layout affects navigation, icons, alignment and mixed-language content, so it should be designed and tested deliberately rather than added as a text-only translation.

Transparency means explaining access before a sensitive action, not only after a problem occurs.

CURRENT POSITION

The prototype demonstrates privacy-aware architecture. It does not claim that moderation policy, identity verification or safeguarding operations are complete.

05 / VISUAL DESIGN PROCESS

Visual Direction

The visual language aims to feel calm, local and trustworthy rather than corporate or overly technical. The interface uses contrast and time-based hierarchy to make the next action easy to understand.

Working design tokens used across the prototype and this portfolio.

Color

Deep teal gives the product a grounded identity; off-white reduces the clinical feeling of pure white; gold marks time and important action; sage supports quieter status and privacy information.

Type and shape

The documentation combines a human serif heading with a clear sans-serif body. In the app, rounded cards and bottom sheets create separation without turning every item into a generic dashboard tile.

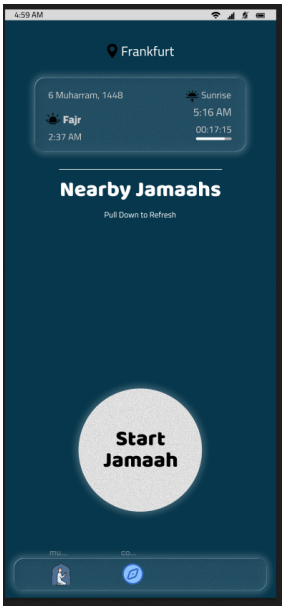
Human over generic

The teacher's feedback challenged the project to move beyond an AI-generated appearance. The response is not simply more decoration. It is to use recognizable local language, real device captures, specific prayer information, carefully written consent and visual details derived from the actual use context.

05 / VISUAL DESIGN PROCESS

Interface Development

An early Figma direction placed the current location, date, prayer, sunrise and countdown inside a translucent hero card. This established the time-sensitive character of the app.



Early home interface. Source: Research/Figma Home interface.png.

What the early design established

The prayer name and countdown were given immediate priority. The location label anchored the interface in the user's surroundings, while the 'Nearby' section suggested discovery below the fold.

What needed refinement

The first screen was visually attractive but still generic and incomplete. It did not yet explain participation, trust, privacy or the relationship between a prayer time and a user-created Jama'ah. Later iterations moved more responsibility into the home list and details bottom sheet.

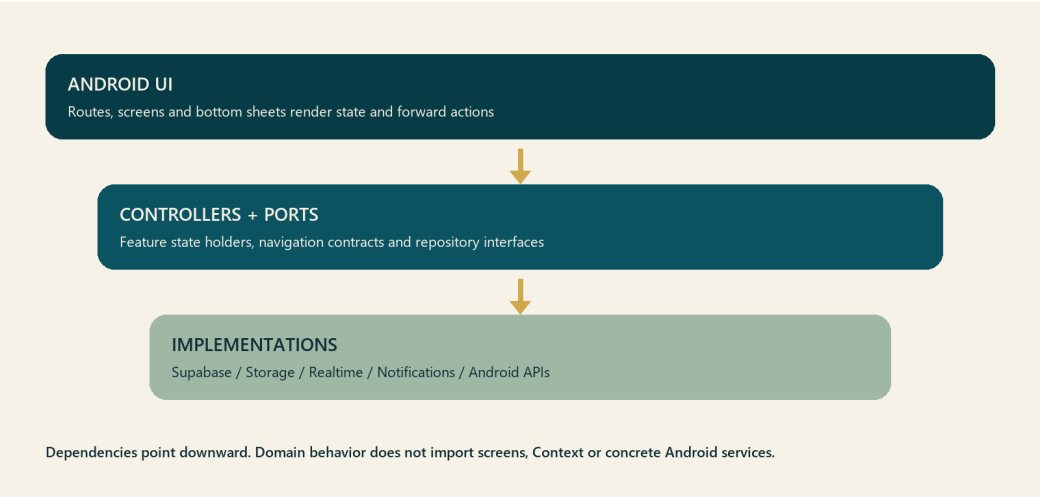
Next visual pass before submission

The visual review compared the interface on a physical phone, checked the clarity of time and action hierarchy, and identified the need for feedback from Muslim users on whether the tone feels familiar, respectful and trustworthy.

06 / TECHNICAL APPROACH AND PROJECT STRUCTURE

Technical Approach

The prototype uses a practical mobile-to-backend architecture. Screen code renders state and forwards actions; repositories and services own data and platform work.



One-directional dependency boundary used by the mobile prototype.

Mobile

Expo React Native and TypeScript provide the Android interface. Expo Router handles routes and deep links, while feature controllers and state holders keep submission and screen behavior outside route files.

Backend

Supabase provides Postgres, PostGIS, authentication, Row Level Security, Realtime and private Storage. Database functions enforce publication, participation and visibility rules.

Platform services

Expo notifications support push registration, Resend SMTP delivers authentication email, and EAS produces a signed standalone APK. These are explicit dependencies, not hidden assumptions.

06 / TECHNICAL APPROACH AND PROJECT STRUCTURE

Project Structure and Ownership

The mobile code is organized around responsibilities so that a change to one behavior does not require editing the entire application entry point.

Mobile owners

- app/: route composition and navigation entry points
- features/: screen-specific UI, controllers and state
- ports/: small interfaces for repositories and platform actions
- services/: Supabase, storage, notification and Android implementations
- providers/: explicit dependency assembly
- navigation/: navigation contracts and deep-link behavior
- theme/ and locales/: shared presentation configuration

Behavior boundaries

- UI renders state and forwards user intent.
- Controllers own transitions such as editing, reviewing and submitting.
- Repositories normalize database success and error results.
- Supabase functions remain the authority for protected data.
- Screen-local state owns the selected bottom sheet, avoiding unnecessary globals.
- The app entry point initializes and assembles dependencies only.

```
Android UI -> controller / state holder -> use case or repository interface
Repository implementation -> Supabase, Storage, Realtime, notifications and Android APIs
```

PRACTICAL ARCHITECTURE

Abstraction was introduced only where it reduced coupling or made behavior testable. The goal was not to reproduce a large enterprise architecture inside a prototype.

07 / AI-ASSISTED WORKFLOW

Safe Change Workflow: Main Rules

The safe-change workflow originated in the earlier Group 6 Felsenmear prototype. I reused it as a practical prompting and verification method while developing Jamaa Next Door.

MAIN POINTS

- Treat each request as a product decision and an architecture change.
- Give each feature an owner, explicit dependencies, a removal path and verification.
- Keep the composition root focused on dependency assembly.
- Preserve behavior unless replacement is explicit.
- Avoid hidden dependencies and shared global state.
- Remove obsolete code, state, UI, imports and comments.
- Verify before stacking additional work.

VERIFICATION

- Run syntax checks.
- Confirm imports resolve.
- Start the local preview server.
- Smoke-test the affected flow.
- Review Git diff for unrelated changes.
- Search for stale references.

GENERAL CHANGE REQUEST

```
I want to change/add/remove [feature].

Desired behavior:
- ...

Keep unchanged:
- ...

Obsolete behavior to remove:
- ...

Safety requirement:
- Keep this loosely coupled.
- Do not add hidden dependencies or shared global state.
- Remove dead code if this replaces an older feature.
- Verify the app still loads and the relevant flow still works.
```

Source: Research/safe-change-workflow.updated.pdf. Prompt wording retained; surrounding explanation adapted to this project.

07 / AI-ASSISTED WORKFLOW

Prompt Templates: Add, Change and Remove

These templates made the requested behavior, protected behavior, module boundary and verification responsibility explicit before implementation began.

ADDING A FEATURE

```
Add [feature].

Behavior:
- ...

Keep unchanged:
- ...

Architecture:
- Keep it loosely coupled.
- Put the feature logic in the smallest appropriate owner module.
- Use explicit dependency injection instead of importing from app.js.
- Do not add global state unless there is no safer option.

Verification:
- Run syntax checks.
- Confirm imports resolve.
- Run the local server smoke test.
- Tell me what changed and what files own the behavior.
```

CHANGING AN EXISTING FEATURE

```
Change [existing feature] so that [new behavior].

Important:
- Preserve [specific behavior].
- Replace [old behavior] if it conflicts.
- Remove obsolete code instead of leaving fallback branches behind.
- Search for stale references after the change.

Keep the code loosely coupled and commit the change separately.
```

REMOVING A FEATURE

```
Remove [feature] completely.

Please remove:
- behavior logic
- unused state fields
- unused UI elements/classes
- unused imports
- stale debug hooks
- stale docs/comments

Do not remove unrelated code. Verify that the app still loads and that nearby features still work.
```

Prompt templates from Research/safe-change-workflow.updated.pdf.

07 / AI-ASSISTED WORKFLOW

Prompt Templates: Fix and Refactor

Bug fixing required a reproduction, a root-cause explanation and a focused regression check. Refactoring was separated from feature change so behavior remained stable.

FIXING A PROBLEM

```
Fix [problem or error].

Observed behavior:
- ...

Expected behavior:
- ...

How to reproduce:
1. ...
2. ...

Safety requirements:
- Identify the root cause before editing.
- Keep the fix in the smallest appropriate owner module.
- Preserve unrelated behavior and public interfaces.
- Use explicit dependencies; do not add hidden coupling or global state.
- Remove any obsolete workaround, dead branch, or stale comment replaced by the
fix.
- Add or update a focused regression check when practical.

Verification:
- Reproduce the problem before the fix when possible.
- Confirm the reproduction no longer fails after the fix.
- Run syntax and import checks.
- Smoke-test the affected flow and nearby behavior.
- Report the root cause, changed files, and verification results.
```

REFACTORING ONLY

```
Refactor [area/module] without changing behavior.

Rules:
- No feature changes.
- Keep public behavior and messages the same.
- Move code into smaller modules with explicit dependencies.
- Remove duplicated or dead code only when it is clearly unused.
- Commit after verification.

For any change that replaces old behavior, include this line:
Also remove anything made obsolete by this change.
```

HOW I APPLIED IT

Observed behavior -> expected behavior -> reproduction -> root cause -> smallest owner -> rebuild -> real-device retest.

EXAMPLE FROM JAMAA NEXT DOOR

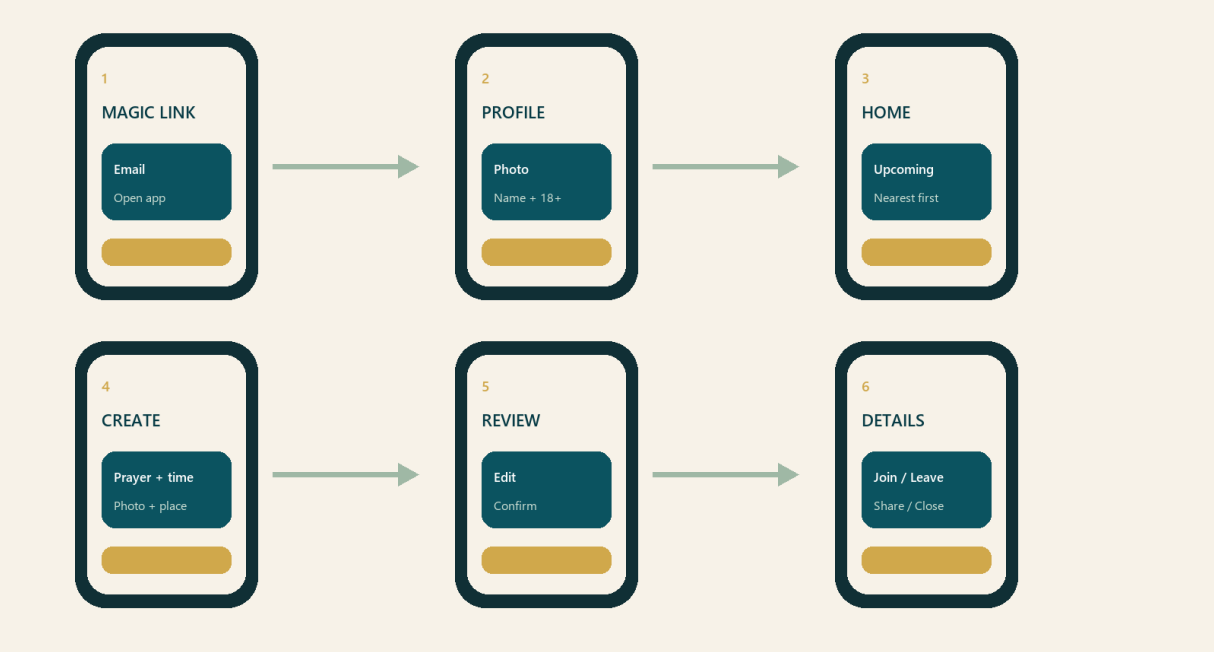
Used for the magic-link callback failure, missing profile submission action, crypto compatibility error and blank Jama'ah details screen.

Prompt templates from Research/safe-change-workflow.updated.pdf.

08 / PROTOTYPE AND FINAL PRODUCT

Prototype and Final Product

The working prototype now demonstrates the complete central journey as a standalone Android application.



Current flow

A user requests a magic link, returns to the app, completes a profile and reaches the Jama'ah home. The user can review upcoming gatherings or create one by selecting the prayer, scheduled time, exact location and required location photo. Confirmation uploads the image, creates the record and returns to a refreshed home list.

Participation

Tapping a home card opens a details bottom sheet. Depending on role and status, the sheet presents Join, Leave, Cancel, Share and Close actions. Shared text contains only approximate area information. Protected location details remain conditional.

Honest implementation status

The APK installs and launches without Expo Go, and the core Supabase-backed workflow has been implemented. This remains a prototype: public release would require final policy decisions, broader user testing, production monitoring and fully reviewed service configuration.

09 / CONCLUSION

Conclusion

Jamaa Next Door began with a personal experience of religious isolation and developed into a working Android prototype with a defined interaction model, backend rules and a traceable design process.

What the project demonstrates

The project connects product design, mobile implementation and responsible system thinking. The main achievement is not only that a Jama'ah can be created. It is that creation, discovery, participation, protected location access and automatic ending are treated as one coherent lifecycle.

What changed in my process

I learned to make my intentions testable. Instead of treating AI output as a finished answer, I used explicit constraints, smaller module ownership, real-device observation and repeated verification. When the result did not match my vision, I improved the problem description and acceptance criteria rather than accepting a generic solution.

Final position

The prototype now communicates the core idea clearly: nearby Muslims should be able to find one another for prayer without exposing more information than the moment requires. The remaining responsibility is to present that idea honestly, validate it with the people it is intended for and ensure that trust is visible in both the interface and the system behind it.

> Software is created for people. In this project, trust, language, timing and visual tone are part of the functionality.